

Import Model v2 Specification

Status: Draft

Scope: Text and AI-assisted import pipeline

Schema target: `import-model-v2`

1) Summary of findings

The legacy import path was vulnerable to node explosion because it treated nearly every non-empty line as a process node and auto-generated linear edges. In practice, long chapter text produced hundreds of nodes, long labels, and poor cluster assignment quality.

Key failure modes:

- Flat parsing (`1 line = 1 node`) with no hierarchy stage.
- No hard node/edge budgets during import.
- Loose AI response contract (plain text only) and no structure validation.
- Limited separation of process steps vs. facts/policies/unclassified snippets.

Import Model v2 addresses this with:

1. Bounded parsing budgets.
2. Deterministic preprocessing and chunking.
3. Hierarchical section + step extraction.
4. Explicit fact/policy/unclassified handling.
5. Structured Gemini response contract and normalization.

2) Design goals

- Prevent import explosions by default.
- Preserve meaningful hierarchy from unstructured text.
- Keep node labels short and readable, move detail into notes.
- Isolate context/facts/policies from executable process flow.
- Keep canonical graph format unchanged for editor compatibility.

3) Import pipeline (v2)

1. ****Preprocess****
 - Normalize OCR line wraps and hyphenation.
 - Drop obvious noise/meta lines (page counters, separators, etc.).
2. ****Chunk / section detect****
 - Detect headings (numbered chapter lines, markdown headers, all-caps titles).
 - Group subsequent lines as section children.
3. ****Classify lines****
 - `process`, `decision`, `fact`, `policy`, `unclassified`.
4. ****Budget + normalize****
 - Enforce max input lines, max process nodes, max annotation nodes, max edges.
 - Truncate long titles and move overflow text to `metadata.notes`.

5. ****Graph build****
 - Build sequence flow from section summary + process/decision lines.
 - Route facts/policies to annotation layer.
 - Aggregate residual noise into an unclassified bucket node.
6. ****Cluster assignment****
 - Apply TOC-seeded cluster mapping after model construction.

4) Budget defaults

```
```json
{
 "maxInputLines": 600,
 "maxProcessNodes": 72,
 "maxAnnotationNodes": 24,
 "maxTotalNodes": 96,
 "maxEdges": 120,
 "maxChildrenPerSection": 12,
 "maxTitleChars": 72,
 "maxNotesChars": 280
}
```

#### ## 5) Canonical mapping rules

- Process-like lines -> canonical `process` or `decision`.
- Section summaries -> canonical `process` with notes.
- Facts/policies -> canonical `annotation` nodes.
- Unclassified snippets -> one or more aggregate `annotation` nodes.

#### Node text policy:

- `label` is short, display-first title.
- `metadata.notes` contains extended detail and overflow text.

#### ## 6) Structured AI contract

Gemini responses are normalized into structured payloads before import:

- TOC mode returns clusters.
- Detail mode returns sections and typed steps.
- OCR mode returns cleaned text.

A strict schema is provided in:

- `docs/import-model.schema.json`
- `docs/import-model.example.json`

#### ## 7) Exportability

The import model payload is:

- JSON-serializable.
- Schema-validated.
- Convertible into deterministic import text for replay.
- Backward-compatible with current canonical model ingestion path.

## ## 8) Evaluation expectations

Import QA now includes additional quality gates:

- `longTitleRate`
- `hierarchyCoverage`
- `factPolicyRecall`
- `nodeBudgetViolationRate`

These metrics complement existing recall/assignment/explosion checks.